

✓ Title : Perform sentiment analysis with a recurrent neural networks RNN

✓ 1) Import Libraries

```
import re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences

np.random.seed(42)
tf.random.set_seed(42)

print('TensorFlow version:', tf.__version__)
```

TensorFlow version: 2.19.0

✓ 2) Load Dataset

```
data_url = 'https://raw.githubusercontent.com/skillcate/sentiment-analysis-with-deep-neural-networks/main/a1_IMDB_Dataset.csv'
df = pd.read_csv(data_url)

print('Dataset shape:', df.shape)
df.head()
```

Dataset shape: (50000, 2)

	review	sentiment
0	One of the other reviewers has mentioned that ...	positive
1	A wonderful little production. The...	positive
2	I thought this was a wonderful way to spend ti...	positive
3	Basically there's a family where a little boy ...	negative
4	Petter Mattei's "Love in the Time of Money" is...	positive

```
df.info()
print('\nMissing values:\n', df.isnull().sum())
```

```
print('\nClass distribution:')
print(df['sentiment'].value_counts())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 2 columns):
#   Column      Non-Null Count  Dtype
---  -
0    review      50000 non-null  object
1    sentiment   50000 non-null  object
dtypes: object(2)
memory usage: 781.4+ KB
```

```
Missing values:
review      0
sentiment    0
dtype: int64
```

```
Class distribution:
sentiment
positive    25000
negative    25000
Name: count, dtype: int64
```

3) Data Pre-processing

```
def clean_text(text):
    text = text.lower()
    text = re.sub(r'<.*?>', ' ', text)      # remove HTML tags
    text = re.sub(r'^a-z\s', ' ', text)     # keep letters and spaces
    text = re.sub(r'\s+', ' ', text).strip() # remove extra spaces
    return text

df['review_clean'] = df['review'].astype(str).apply(clean_text)

# Encode labels: positive -> 1, negative -> 0
label_map = {'negative': 0, 'positive': 1}
df['label'] = df['sentiment'].map(label_map)

df[['review', 'review_clean', 'sentiment', 'label']].head()
```

	review	review_clean	sentiment	label
0	One of the other reviewers has mentioned that ...	one of the other reviewers has mentioned that ...	positive	1
1	A wonderful little production. The...	a wonderful little production the filming tech...	positive	1
2	I thought this was a wonderful way to spend ti...	i thought this was a wonderful way to spend ti...	positive	1
3	Basically there's a family where a little boy ...	basically there s a family where a little boy ...	negative	0
4	Petter Mattei's "Love in the Time of Money" is...	petter mattei s love in the time of money is a...	positive	1

```
# Train-test split
X_train_text, X_test_text, y_train, y_test = train_test_split(
    df['review_clean'],
    df['label'],
    test_size=0.2,
    random_state=42,
    stratify=df['label']
)

print('Train size:', X_train_text.shape[0])
print('Test size :', X_test_text.shape[0])
```

```
Train size: 40000
Test size : 10000
```

```
# Tokenization and sequence padding
MAX_WORDS = 20000
MAX_LEN = 200

tokenizer = Tokenizer(num_words=MAX_WORDS, oov_token='<OOV>')
tokenizer.fit_on_texts(X_train_text)

X_train_seq = tokenizer.texts_to_sequences(X_train_text)
X_test_seq = tokenizer.texts_to_sequences(X_test_text)

X_train_pad = pad_sequences(X_train_seq, maxlen=MAX_LEN, padding='post', truncating='post')
X_test_pad = pad_sequences(X_test_seq, maxlen=MAX_LEN, padding='post', truncating='post')

print('X_train_pad shape:', X_train_pad.shape)
print('X_test_pad shape :', X_test_pad.shape)
```

```
X_train_pad shape: (40000, 200)
X_test_pad shape : (10000, 200)
```

4) Build RNN Model

```
EMBED_DIM = 128

model = keras.Sequential([
    layers.Input(shape=(MAX_LEN,)),
    layers.Embedding(input_dim=MAX_WORDS, output_dim=EMBED_DIM),
    layers.SimpleRNN(64, return_sequences=True),
    layers.Dropout(0.3),
    layers.SimpleRNN(32),
    layers.Dropout(0.3),
```

```

        layers.Dense(16, activation='relu'),
        layers.Dense(1, activation='sigmoid')
    ])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	Param #
embedding (Embedding)	(None, 200, 128)	2,560,000
simple_rnn (SimpleRNN)	(None, 200, 64)	12,352
dropout (Dropout)	(None, 200, 64)	0
simple_rnn_1 (SimpleRNN)	(None, 32)	3,104
dropout_1 (Dropout)	(None, 32)	0
dense (Dense)	(None, 16)	528
dense_1 (Dense)	(None, 1)	17

Total params: 2,576,001 (9.83 MB)

Trainable params: 2,576,001 (9.83 MB)

```

# Optional network graph visualization
try:
    keras.utils.plot_model(model, show_shapes=True, show_layer_names=True, dpi=72)
    print('Model graph generated successfully (check notebook output/files).')
except Exception as e:
    print('Model graph could not be generated in this environment:', e)

```

Model graph generated successfully (check notebook output/files).

5) Train the Model

```

callbacks = [
    keras.callbacks.EarlyStopping(monitor='val_loss', patience=2, restore_best_weights=True)
]

history = model.fit(
    X_train_pad,
    y_train,
    validation_split=0.2,
    epochs=8,
    batch_size=128,
    callbacks=callbacks,
    verbose=1
)

```

```

Epoch 1/8
250/250 ————— 55s 207ms/step - accuracy: 0.5041 - loss: 0.6977 - val_accuracy: 0.4989 - val_loss: 0.6938
Epoch 2/8
250/250 ————— 51s 203ms/step - accuracy: 0.5050 - loss: 0.6936 - val_accuracy: 0.4997 - val_loss: 0.6937
Epoch 3/8
250/250 ————— 84s 211ms/step - accuracy: 0.5043 - loss: 0.6978 - val_accuracy: 0.4991 - val_loss: 0.6940
Epoch 4/8
250/250 ————— 81s 205ms/step - accuracy: 0.5041 - loss: 0.6941 - val_accuracy: 0.5045 - val_loss: 0.6934
Epoch 5/8
250/250 ————— 81s 202ms/step - accuracy: 0.4994 - loss: 0.6942 - val_accuracy: 0.5031 - val_loss: 0.6933
Epoch 6/8
250/250 ————— 81s 198ms/step - accuracy: 0.5048 - loss: 0.6933 - val_accuracy: 0.5035 - val_loss: 0.6932
Epoch 7/8
250/250 ————— 83s 203ms/step - accuracy: 0.5081 - loss: 0.6932 - val_accuracy: 0.5038 - val_loss: 0.6932
Epoch 8/8
250/250 ————— 83s 208ms/step - accuracy: 0.5041 - loss: 0.6932 - val_accuracy: 0.5049 - val_loss: 0.6931

```

```

# Training curves
fig, ax = plt.subplots(1, 2, figsize=(12, 4))

```

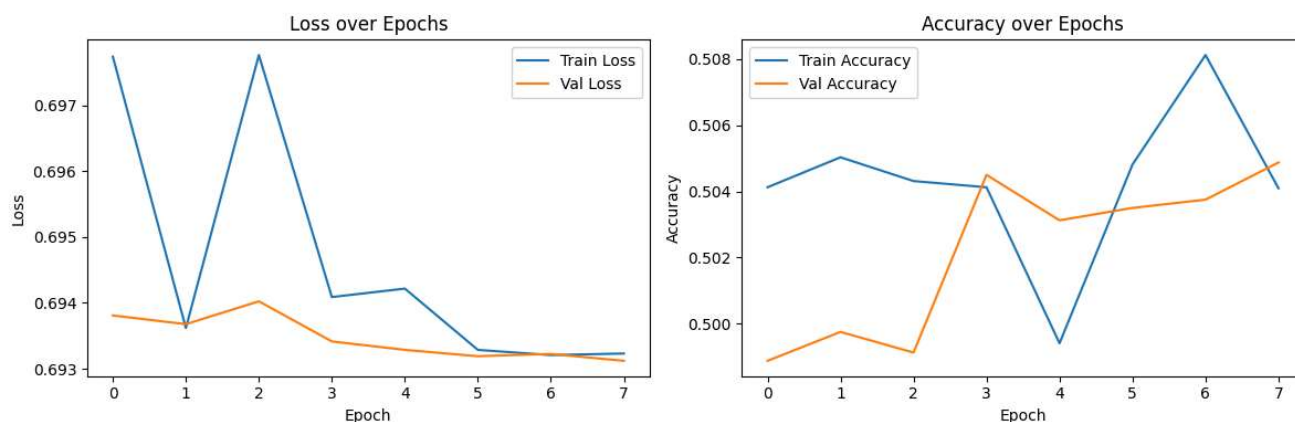
```

ax[0].plot(history.history['loss'], label='Train Loss')
ax[0].plot(history.history['val_loss'], label='Val Loss')
ax[0].set_title('Loss over Epochs')
ax[0].set_xlabel('Epoch')
ax[0].set_ylabel('Loss')
ax[0].legend()

ax[1].plot(history.history['accuracy'], label='Train Accuracy')
ax[1].plot(history.history['val_accuracy'], label='Val Accuracy')
ax[1].set_title('Accuracy over Epochs')
ax[1].set_xlabel('Epoch')
ax[1].set_ylabel('Accuracy')
ax[1].legend()

plt.tight_layout()
plt.show()

```



6) Evaluate Model

```

test_loss, test_acc = model.evaluate(X_test_pad, y_test, verbose=0)
print(f'Test Loss : {test_loss:.4f}')
print(f'Test Accuracy: {test_acc:.4f}')

```

Test Loss : 0.6932
Test Accuracy: 0.5006

```

y_prob = model.predict(X_test_pad, verbose=0).ravel()
y_pred = (y_prob >= 0.5).astype(int)

print('Accuracy score:', accuracy_score(y_test, y_pred))
print('\nClassification Report:\n')
print(classification_report(y_test, y_pred, target_names=['negative', 'positive'], digits=4))

```

Accuracy score: 0.5006

Classification Report:

	precision	recall	f1-score	support
negative	0.5011	0.2746	0.3548	5000
positive	0.5004	0.7266	0.5927	5000
accuracy			0.5006	10000
macro avg	0.5008	0.5006	0.4737	10000
weighted avg	0.5008	0.5006	0.4737	10000

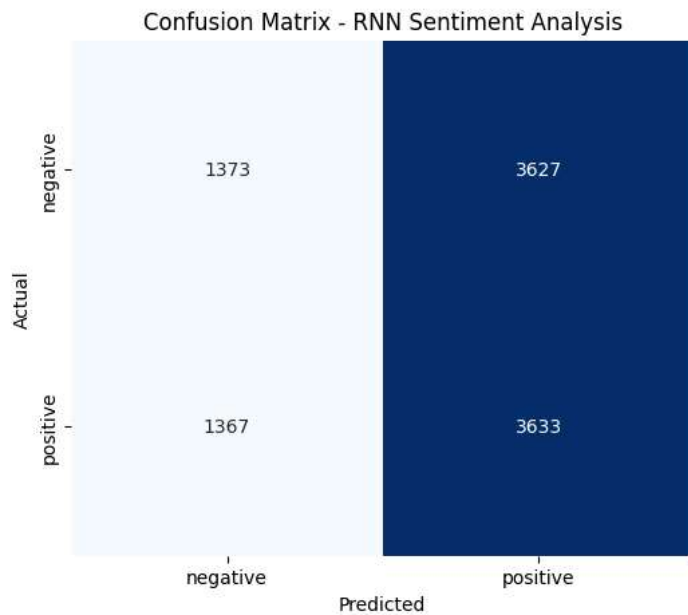
```

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
            xticklabels=['negative', 'positive'],
            yticklabels=['negative', 'positive'])
plt.title('Confusion Matrix - RNN Sentiment Analysis')

```

```
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.show()
```



7) Try Custom Predictions

```
def predict_sentiment(text):
    text_clean = clean_text(text)
    seq = tokenizer.texts_to_sequences([text_clean])
    pad = pad_sequences(seq, maxlen=MAX_LEN, padding='post', truncating='post')
    prob = model.predict(pad, verbose=0)[0][0]
    label = 'positive' if prob >= 0.5 else 'negative'
    return label, float(prob)

samples = [
    'This movie was fantastic, emotional and beautifully acted.',
    'The film was boring, too long, and not worth watching.'
]

for s in samples:
    label, prob = predict_sentiment(s)
    print(f'Text: {s}')
    print(f'Predicted Sentiment: {label} (probability of positive={prob:.4f})\n')
```

Text: This movie was fantastic, emotional and beautifully acted.
 Predicted Sentiment: positive (probability of positive=0.5023)

Text: The film was boring, too long, and not worth watching.
 Predicted Sentiment: positive (probability of positive=0.5023)

